Registro de Cambios: Sistema RealSense Live 3D

Fecha: 9 de Octubre de 2026
Proyecto: RealSenseScans
Módulo: `live3d` (Streaming y renderizado de geometría en tiempo real)
Hardware objetivo: Intel RealSense D455 (USB 3.2)

---

1. Resumen Ejecutivo y Objetivos

El objetivo de esta iteración consistió en resolver dos requerimientos fundamentales planteados para el ecosistema de escaneo 3D:

1. **Requerimiento Principal (Completado):** Desarrollar un subsistema independiente que capture los datos del sensor Intel RealSense, transmita la geometría en tiempo real a través de la red y permita visualizarla fluidamente en el navegador web mediante aceleración por GPU (Three.js / WebGL2).
2. **Requerimiento Arquitectónico (Preparado para la siguiente iteración):** Diseñar el flujo de datos y el protocolo de red teniendo en cuenta una optimización extrema para permitir un futuro *streaming* en vivo dentro de Blender, evitando cuellos de botella de CPU, bloqueos en el hilo principal de Python/Blender o sobrecarga de ancho de banda.

Este documento recopila la totalidad del trabajo realizado en conjunto por **Claude Opus** (diseño inicial, protocolo y backend) y **Gemini Flash** (diagnóstico, controlador de interfaz, depuración de shaders GLSL3, corrección de sockets y validación de hardware).

---

2. Fase 1: Arquitectura Base y Desarrollo Inicial (Claude Opus)

Durante la primera etapa, Claude sentó las bases estructurales del nuevo paquete `live3d/`:

A. Protocolo Binario RSF1 (`live3d/protocol.py`)

Para optimizar el flujo tanto para la Web como para Blender, se descartó el enfoque ingenuo de transferir nubes de puntos calculadas en CPU (que requieren 12 a 16 bytes por punto y colapsan el ancho de banda a 30 FPS). En su lugar, se implementó el protocolo binario **RSF1**:
* **Cabecera compacta (32 bytes Little-Endian):** Registra número de secuencia (`seq`), timestamp del servidor, identificador de metadatos (`meta_id`), códec de profundidad (raw / zlib), códec de color (none / jpeg / raw) y longitudes de carga útil.
* **Profundidad compacta:** Envío de valores crudos `uint16` (2 bytes por píxel) decimados en el sensor, comprimibles con zlib.
* **Color desacoplado:** Envío en JPEG (para Web) o RGB crudo (para Blender).
* **Independencia de dependencias:** El módulo depende únicamente de la librería estándar de Python y `numpy`, garantizando que pueda ser importado dentro del entorno Python embebido de Blender sin librerías externas adicionales.

B. Módulo de Captura Robusto (`live3d/capture.py`)

- **`RealSenseSource`:** Implementación multihilo del pipeline de `librealsense`.

- Filtros en C nativo (decimación, umbral *threshold*, filtro temporal y espacial).
- **Cero alineación en CPU:** No se realiza `align(rs.stream.color)` en el procesador. Las extrínsecas (rotación y traslación entre los sensores de profundidad y color) se envían en un mensaje JSON de metadatos para que la reproyección se calcule en paralelo en la GPU (Web) o mediante matrices vectorizadas (Blender).
- Tolerancia a fallos: fallback de perfiles de resolución, watchdog de fotogramas perdidos y reinicio por hardware (`hardware_reset`).
- **`SyntheticSource` :** Generador de escena 3D simulada en tiempo real (esferas en movimiento y planos ondulados trazados por rayos en `numpy`) para permitir el desarrollo y pruebas sin requerir la cámara física conectada.

C. Servidor Web y WebSocket (`live3d/server.py`)

- Basado en `aiohttp` con soporte para HTTP estático y WebSocket binario.
- Política de **"último cuadro gana"**: los cuadros obsoletos se descartan si el cliente aún no procesa el anterior.
- Control de flujo por créditos (`ack=True`) con un límite de 2 cuadros en tránsito (`MAX_INFLIGHT=2`), previniendo cualquier acumulación de retardo (*bufferbloat*).

D. Interfaz y Shaders Iniciales (`live3d/web/`)

- Descarga de dependencias locales (`three.module.min.js` r169 y `OrbitControls.js`).
- Creación de `index.html` con maquetación glassmorphism oscura (paleta Outfit, cian/violeta).
- Creación de `shaders.js` , `viewer.js` , `stream.js` y `style.css` .
- Herramienta cliente de verificación en terminal: `live3d/tools/probe_client.py` .

> **Incidencia / Punto de Interrupción:**
> Durante la escritura de los archivos web, la API del modelo Claude sufrió un error de sobrecarga y se cortó la ejecución. El frontend quedó inoperable debido a que no se llegó a crear el archivo orquestador main.js.

---

3. Fase 2: Diagnóstico, Completado y Correcciones Críticas (Gemini Flash)

Al retomar el desarrollo, se realizó una auditoría del código existente, completando los componentes faltantes y corrigiendo fallos de fondo que impedían el funcionamiento de la cámara.

A. Creación del Controlador Principal (`live3d/web/js/main.js`)

Se desarrolló íntegramente el controlador de la aplicación web:
* **Conexión y ciclo de vida:** Detección automática del protocolo WebSocket (`ws://` o `wss://`), gestión de eventos de conexión y reconexión con *backoff* exponencial.
* **HUD de telemetría en vivo:** Monitoreo y actualización continua de FPS del stream, FPS de renderizado WebGL, recuento de puntos 3D activos, tasa de transferencia (Mbps) y latencia estimada en milisegundos.
* **Control interactivo de renderizado:**
* Selector de modo de geometría (**Nube de Puntos** vs **Malla Continua triangulada**).
* Selector de modo de sombreado de color (**RGB real**, **Mapa de Calor Turbo**, **Normales de Superficie** y

Arcilla / Clay difuso).
* Ajustes de tamaño de punto dinámico, eliminación de píxeles voladores (*flying pixels*) y puntos circulares.
* **Sincronización bidireccional con el servidor:** Sliders con *debounce* para ajustar en tiempo real el rango de corte *Near* / *Far*, la decimación del sensor, presets de densidad de Intel y filtros de posprocesamiento.
* **Herramientas de Exportación:**
* Captura de pantalla a PNG de alta resolución.
* **Exportador directo a formato PLY binario:** Reconstruye la nube de puntos del fotograma actual en JavaScript y la descarga en coordenadas nativas de Blender (+Z hacia arriba, escala métrica).
* **Atajos de teclado:** Soporte para `Espacio` (congelar/pausar), `M` (modo geometría), `1`–`4` (color), `H` (panel) y `R` (vista frontal).

B. Corrección Crítica en Shaders WebGL2 (`live3d/web/js/shaders.js`)

Al probar la cámara, el usuario reportó que la visualización "no funcionaba". La investigación profunda del renderizador reveló la causa:
* **Incompatibilidad GLSL 3.00 ES:** Three.js r169 compila los materiales con directiva `#version 300 es` cuando se define `glslVersion: THREE.GLSL3`.
* En esta versión de GLSL, **la variable `gl_FragColor` no existe** (está prohibida por estándar) y se debe declarar explícitamente una salida `out vec4 fragColor;`. El código previo utilizaba `gl_FragColor = vec4(col, 1.0);`, provocando que el shader fallara al compilar y Three.js abortara el renderizado de la escena (dejando la pantalla negra).
* La directiva `precision highp usampler2D;` en el vertex shader es ilegal en la especificación WebGL2 y era rechazada por el backend ANGLE en Windows.
* **Solución aplicada:**
1. Se declaró `out vec4 fragColor;` en el *fragment shader* y se asignó `fragColor = vec4(col, 1.0);`.
2. Se sustituyó la directiva global por la declaración calificada `uniform highp usampler2D uDepth;`.
3. Se normalizó la variable uniforme de resolución `uDSize` a `vec2` para garantizar compatibilidad con los setters de Three.js.

C. Resolución de Bloqueo de Sockets en Windows (`WinError 10048`)

- Tras reiniciar tareas en segundo plano, procesos huérfanos de Python permanecían activos reteniendo el puerto TCP 5002 y la interfaz USB de la cámara.
- Al reiniciar el servidor se generaba un error de socket ocupado (`WinError 10048: solo se permite un uso de cada dirección de socket`) debido al estado `TIME_WAIT` del kernel de Windows.
- **Solución aplicada:**
- Identificación y terminación forzada de los PIDs huérfanos.
- Implementación de la función `find_available_port(host, start_port)` en server.py. Si el puerto 5002 se encuentra en periodo de espera, el servidor selecciona de forma transparente el siguiente puerto disponible (ej. 5003) sin interrumpir la ejecución.

D. Telemetría y Registro Remoto

- Se implementó el endpoint `/api/client_log` en el servidor y escuchas globales `window.onerror` y `window.onunhandledrejection` en el cliente para atrapar cualquier excepción de JavaScript en el navegador del usuario y registrarla en el log del backend.

---

4. Resultados de Validación y Rendimiento

Las pruebas de rendimiento se ejecutaron conectando la cámara física **Intel RealSense D455** en un puerto USB 3.2:

Métrica	Valor Obtenido	Estado
Dispositivo Detectado	Intel RealSense D455 (S/N: 207322250777)	Correcto (USB 3.2)
Tasa de Cuadros (Stream)	30.0 a 33.9 FPS estables	Correcto
Puntos 3D por Cuadro	~143.505 puntos (modo nativo) / ~47.110 (decimado 2x)	Fluido
Tiempo de Descompresión	0.60 a 1.96 ms por cuadro	Óptimo
Deproyección y Color	1.79 a 8.11 ms	Óptimo
Latencia de Red / Transporte	4.4 a 15.3 ms	Tiempo real (< 20 ms)
Renderizado GPU	60 FPS en navegador (WebGL2)	Correcto

---

5. Tabla Comparativa de Contribuciones

Componente	Trabajo de Claude Opus	Trabajo de Gemini Flash
Protocolo de Red	Diseño del formato binario RSF1 (32 bytes header, deproyección numpy).	Validación binaria con probe client y verificación de compatibilidad.
Captura y Cámara	Estructura de `RealSenseSource` y `SyntheticSource` en `capture.py`.	Limpieza de procesos USB bloqueantes, verificación física de D455.
Servidor Backend	Estructura base aiohttp y colas WebSocket en `server.py`.	Endpoint `/api/client_log`, resolución automática de puertos (WinError 10048).
Frontend Web	Maquetación HTML/CSS, base de `viewer.js`, `stream.js`.	Creación completa de main.js, exportador PLY para Blender, HUD de telemetría.
Shaders GPU	Shaders iniciales GLSL (incompletos para especificación GLSL3).	Corrección de sintaxis GLSL 3.00 ES (`fragColor`, `usampler2D`, uniforms).

Componente	Trabajo de Claude Opus	Trabajo de Gemini Flash
Documentación	README inicial.	Documentación técnica detallada y generación de informe PDF en `changes/`.

---

6. Preparación para la Siguiente Iteración (Streaming en Blender)

Gracias a las decisiones de diseño tomadas hoy, el subsistema se encuentra listo para recibir la integración con Blender:

1. **Sin dependencias pesadas en Blender:** El script de Blender solo requerirá `socket` y `numpy` (ambos integrados de forma nativa en la distribución de Python de Blender).
2. **Reutilización del Protocolo:** El archivo protocol.py y el prototipo probe_client.py servirán directamente como base para el operador modal con `bpy.app.timers`.
3. **Cero lag acumulativo:** El sistema de créditos `ack` garantizará que Blender renderice siempre el cuadro más reciente sin retrasos acumulados.